

API Pentesting Report

Task – API Penetration Testing

Target - OWASP crAPI

Name: Ravipudi Yoganandh
Enrollement No: 2203031260195
Roll No: 125

Introduction

Scope

Identified Vulnerabilities List

- BOLA (Broken Object Level Authorization)
- Broken User Authentication
- Excessive Data Exposure
- Mass Assignment
- Injection (SQL Injection)
- Security Misconfiguration

Recommendations

Introduction:-

This assessment involves a API Penetration Test focused on identifying security weaknesses within the crAPI (Completely Ridiculous API) environment. Following the industry-recognized OWASP Web Security Testing Guide , this test evaluates the security posture of the application's communication layer from an external attacker's perspective.

Unlike traditional web testing that examines the user interface, API pentesting targets the "pipes" or logic that facilitate data exchange between the client and the server. The goal is to uncover common yet critical vulnerabilities such as Injection, Broken Access Control, and Security Misconfigurations. By simulating real-world attack scenarios through manual testing and behavioral analysis, this report provides actionable recommendations to prevent data compromise or unauthorized account takeover.

Scope:-

The scope of this engagement is limited to the functionalities and endpoints provided by the crAPI lab environment. The assessment focuses on the following key areas:

- **Authentication & Authorization:** Evaluating mechanisms for user login and resource access to identify flaws like Broken Object Level Authorization (BOLA) or admin login bypass.
- **Input Handling & Sanitization:** Testing endpoints for vulnerabilities such as SQL Injection (SQLi) and Cross-Site Scripting (XSS) by manipulating API request bodies and parameters.
- **Session Management:** Analyzing JWT (JSON Web Tokens) and local storage to identify weak hashing (MD5) or token abuse potential.
- **Business Logic & Information Disclosure:** Identifying flaws that allow for repetitive registration, confidential document exposure, or sensitive metadata leakage.
- **Technical Environment:** The testing environment includes the use of Docker for application deployment , Postman for API request orchestration, and Burp Suite for traffic interception.

Environment Setup for OWASP crAPI API Pentesting:-

Operating System - Linux (Kali Linux)

Containerization Platform – Docker

Docker Installation & Configuration

- Step 1: Install Docker

```
(yogi@kali)-[~]
$ sudo apt-get install docker.io -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Solving dependencies... Done
The following packages were automatically installed and are no longer required:
  apg firmware-ti-connectivity gir1.2-girepository-2.0
  gir1.2-javascriptcoregtk-6.0 gir1.2-webkit-6.0 gkbd-caplet
  gnome-backgrounds ibus-gtk icu-devtools libabsl20230802 libavformat60
  libdisplay-info1 libdnnl3 libfuse3-3 libgav1-1 libgdata-common libgdata22
  libgirepository-1.0-1 libglapi-mesa libgnome-rr-4-2t64 libgnomekbd-common
  libgnomekbd8 libicu-dev libinstpatch-1.0-2 libjavascriptcoregtk-6.0-1
  libjs-jquery-ui liblbfgsb0 liblua5.2-0 libmjpegutils-2.1-0t64 libmozjs-128-0
  libmpeg2encpp-2.1-0t64 libmpeg2-2.1-0t64 libnet1 libnghttp3-3 libopenh264-7
```

- Step 2 : Verify Docker Installation

```
(yogi@kali)-[~]
$ docker --version
Docker version 27.5.1+dfsg4, build cab968b3

(yogi@kali)-[~]
$
```

Docker Hub Authentication:

- Step 1: Login to Docker Hub

```
(yogi@kali)-[~/crAPI/deploy/docker]
$ docker login -u ryogi2004
Password:
WARNING! Your password will be stored unencrypted in /home/yogi/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credential-stores

Login Succeeded
```

Downloading OWASP crAPI:

- Step 1: Clone crAPI Repository:

```
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 12

(yogi@kali)-[~]
$ git clone https://github.com/OWASP/crAPI.git
Cloning into 'crAPI'...
remote: Enumerating objects: 7032, done.
remote: Counting objects: 100% (134/134), done.
remote: Compressing objects: 100% (110/110), done.
remote: Total 7032 (delta 38), reused 48 (delta 20), pack-reused 6898 (from 4)
Receiving objects: 100% (7032/7032), 8.61 MiB | 2.69 MiB/s, done.
Resolving deltas: 100% (3816/3816), done.
```

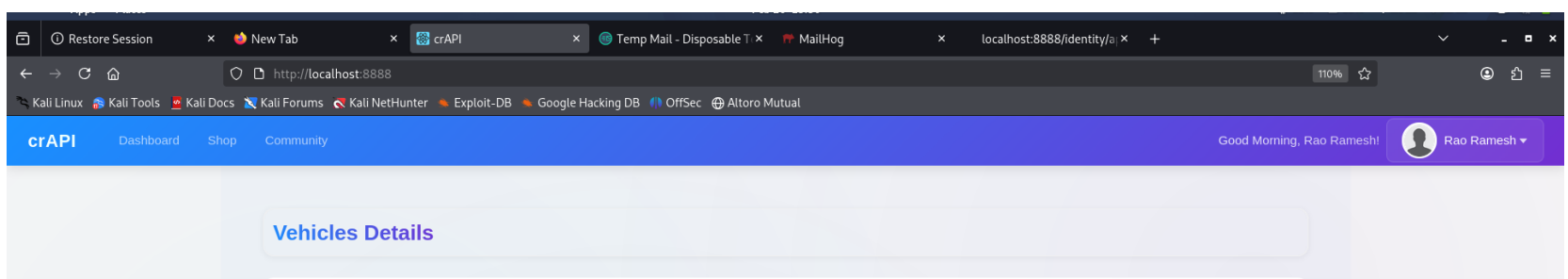
Running crAPI Using Docker Compose:

- Step 1: Start crAPI Containers:-
Cmd – “sudo docker compose up -d ”

```
[+] Running 13/14
✔ Network docker_default          Created           0.5s
✔ Volume "docker_mongodb-data"    Created           0.0s
✔ Volume "docker_postgresql-data" Created           0.0s
✔ Volume "docker_chromadb-data"   Created           0.0s
✔ Container postgresdb            Healthy          24.4s
✔ Container mongodb               Healthy          24.4s
✔ Container api.mypremiumdealership.com Started           8.6s
✔ Container mailhog                Healthy          23.1s
✔ Container chromadb               Healthy          24.4s
✔ Container crapi-identity         Waiting          64.7s
✔ Container crapi-community        Created           0.3s
✔ Container crapi-chatbot          Created           0.3s
✔ Container crapi-workshop         Created           0.2s
✔ Container crapi-web              Created           0.2s
```

Accessing crAPI Application-

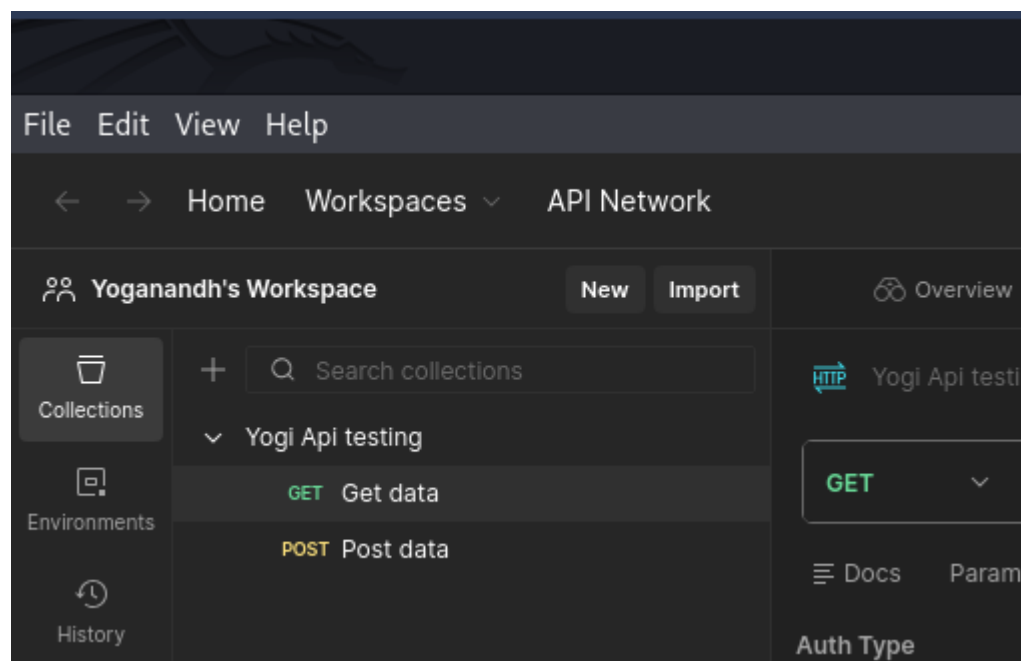
1. Open a web browser (Chrome, Firefox, or Edge recommended)
2. Enter <http://localhost:8888> in the address bar.



Postman Installation (Desktop Version)

-Postman was installed manually via CLI:

Step 1: Extract Postman



Identified Vulnerabilities:

VULN-02: Broken Authentication:

Owasp Name - Broken Authentication

- POST Login API
- Endpoint : POST /identity/api/auth/login

Severity: High

Description:

The login API generates a JWT token upon successful authentication. The token is used to authorize access to protected endpoints such as the dashboard API. During testing, the token was manually extracted and reused in Postman to access protected resources. The application relies entirely on the provided token for authentication without additional session validation controls. If token validation is weak (e.g., missing expiration checks, improper signature validation, or lack of revocation), attackers may reuse stolen tokens to access user accounts.

Impact:

- Unauthorized account access
- Session hijacking
- Account takeover
- Persistent unauthorized API access

